STERLING · SBG FUNDING AI GUIDE

Study guide

docs/STUDY_GUIDE.md · October 29, 2026

This guide gets you from zero to explaining every part of Sterling in about three and a half hours. It has five parts:

1. The map: what the system does and where each piece lives
2. Concepts: LLMs, tool use, agents, RAG, embeddings, BM25, hybrid search, guardrails, evals
3. Trace two requests through the code
4. The lending domain: what SBG sells and the vocabulary
5. Study plan and self-check

Keep the app running while you read (`npm run build && npm start`) and open "Under the hood" often. Every concept below shows up in that trace.

---

1. The map (10 minutes)

Sterling answers one question for a business owner: *"What financing fits me, do I qualify, what would it cost, and who do I talk to?"*

Every message goes through the same pipeline:

```
message ─► guardrails ─► (sensitive? ─► human handoff)
                  └──► Claude with tools ─► retrieval (facts)
                                         ├─► eligibility + calculator (decisions, numbers)
                                         ├─► profile update (what the user told us)
                                         └─► escalate (when a person is better)
        ─► the guided flow decides what card to show next
```

The idea that ties it together: **code decides, the model explains.** Anything that must be correct and consistent (eligibility, dollar figures, question order, routing of sensitive messages) is ordinary, tested code. The model does what code can't: understand free text, pick the right tool, and explain results in plain English, grounded in retrieved documents.

Where things live:

You want to understand…	Read
One turn, end to end	`lib/chat/orchestrator.ts`
The Claude loop	`lib/agent/agent.ts` , `lib/agent/tools.ts` , `lib/agent/prompt.ts`

You want to understand…	Read
RAG	`scripts/ingest.ts`, `lib/rag/chunker.ts`, `lib/rag/embedder.ts`, `lib/rag/bm25.ts`, `lib/rag/retriever.ts`
Decisions	`lib/engine/eligibility.ts`, `lib/engine/estimator.ts`, `lib/engine/flow.ts`
Safety	`lib/engine/guardrails.ts`
Handoff	`lib/handoff/escalate.ts`, `lib/handoff/summary.ts`, `components/inbox/Inbox.tsx`
What the browser does	`components/chat/useChat.ts` and the components it feeds

---

2. Concepts

2.1 What an LLM call actually is

A large language model predicts the next token (a word piece, about ¾ of a word on average) given everything before it. An API call sends:

- a **system prompt**: standing instructions (`SYSTEM_PROMPT` in `lib/agent/prompt.ts`);
- a list of **messages**, alternating user and assistant;
- optionally, **tool definitions**.

It returns the model's next message.

Three consequences drive a lot of the design:

1. **The model is stateless.** It remembers nothing between calls; you resend the conversation every time. That's why `ClientState.history` exists, and why it's capped at 24 turns: everything you send costs tokens and time.
2. **The context window is finite.** Only so many tokens fit in one request. You can't paste all of SBG's website into every call, which is one reason for RAG (section 2.4).
3. **The model can be confidently wrong** ("hallucination"). It produces plausible text, not verified facts. Hence grounding (RAG plus citations) and deterministic tools for anything that must be correct.

Thinking and effort. Sterling's model thinks before answering ("adaptive thinking"): it decides how much to reason based on the question. The `effort` setting (`low` here) caps how much. Low keeps chat replies fast. During one turn, thinking blocks come back with a cryptographic `signature`, and the loop must send them back unchanged when it continues after a tool call. `tests/agent.test.ts` checks this.

Prompt caching. The system prompt and tool definitions are identical on every call, so they're marked `cache_control: ephemeral`. After the first call, the API reuses its processed copy of that prefix: cheaper and faster. That's why per-turn details (profile, time, next question) go in an `<app_state>` block on the

newest user message instead of the system prompt. Changing the system prompt every turn would invalidate the cache.

Streaming. Instead of waiting for the full answer, the API sends events as the model generates (server-sent events). `agent.ts` forwards each text delta to the browser immediately. Time to the first word matters far more to how fast a chat feels than total time.

2.2 Tool use (function calling)

You describe tools to the model: a name, a description and a JSON Schema for the input. The model can reply with a **tool_use** block (`{name: "search_knowledge_base", input: {query: "draw fee"}}`) instead of, or as well as, text. **Your code** runs the tool and sends back a **tool_result**. The model never executes anything itself; it only asks.

Sterling's five tools (`lib/agent/tools.ts`):

Tool	What it does	Why it's a tool
`search_knowledge_base`	Hybrid retrieval, returns numbered passages	Facts must come from SBG's content
`update_business_profile`	Saves details the user mentioned	Turns free text into structured data the flow and engine use
`check_eligibility`	Runs the deterministic eligibility engine	Decisions must be consistent and correct
`estimate_funding`	Runs SBG's calculator model	The model must never do the math
`escalate_to_human`	Opens a handoff ticket	A human-in-the-loop decision the model can make

Every tool input is validated with zod before it runs. If the model sends something invalid, it gets an `is_error` result explaining what was wrong and can try again. Never trust model output as input to code without validation, just as you wouldn't trust a form submission.

2.3 The agent loop

An "agent" here is a loop around tool use (`runAgent` in `lib/agent/agent.ts`):

```
messages = history + newest user message (with <app_state>)
repeat up to 6 times:
    response = call Claude(system, tools, messages)       # streamed
    if response has no tool calls: done (it answered)
    run each tool call -> results
    messages += [assistant response, user message with all tool results]
```

Details that matter:

- **A step limit** (6) prevents an infinite loop.

- **All tool results go back together** in one user message, in the order the calls were made.
- **Refusals** (`stop_reason: "refusal"`): stop, and never run tools from a declined response.
- **Truncation** (`stop_reason: "max_tokens"`): a tool call cut off mid-input isn't run.
- **No forced tool choice.** The code doesn't force a tool call (`tool_choice`); with thinking enabled that isn't allowed, and the prompt plus good tool descriptions are enough.

2.4 RAG: retrieval-augmented generation

The problem: the model doesn't know SBG's current products, terms or policies, and they change. **The fix:** at question time, *retrieve* the relevant passages from SBG's own content and put them in the context, then *generate* an answer from them, with citations.

RAG has two halves.

Offline (ingest), `npm run ingest`:

1. **Parse** the PDF into text with structure (headings, paragraphs, bullets). `lib/rag/pdf.ts` recovers headings from font sizes.
2. **Chunk** into passages. A chunk is the unit you retrieve. Too big, and it's vague and wastes context. Too small, and it loses meaning. Sterling uses *structure-aware* chunking: never cross a heading, at most 150 words, and repeat the last bullet of one chunk at the start of the next (overlap) so lists keep context. 150 words is about 200 tokens, inside the embedding model's 256-token limit, so nothing gets silently cut off.
3. **Add context to each chunk.** The embedded text is `"Section > Title\nPassage"`, so "up to 90% of value" knows it's about equipment financing.
4. **Embed** each chunk (section 2.5) and save everything to `data/kb-index.json`.

Online (query), `search()` in `lib/rag/retriever.ts`:

1. Embed the question.
2. Score every chunk two ways: dense (cosine) and BM25 (keywords).
3. Fuse the two rankings with RRF and take the top k.
4. Check confidence: if even the best match is weak, say "I don't know" instead of answering.
5. Return passages numbered [1], [2], [3]. The model cites them, and the UI shows only the sources the answer actually cited.

2.5 Embeddings and cosine similarity

An **embedding model** turns text into a vector: here, 384 numbers. It was trained so that texts with similar meaning get vectors pointing in similar directions. "I can't make payroll" and "cover payroll gaps" share almost no words, but their vectors are close.

Cosine similarity measures the angle between two vectors:

```
cos(a, b) = (a · b) / (|a| × |b|)
```

1 means the same direction, 0 means unrelated. Sterling normalizes every vector to length 1, so cosine reduces to a dot product: multiply matching positions and add them up. With 105 chunks, comparing the query against every vector takes well under a millisecond. Brute force is fine. At millions of chunks you'd use an approximate nearest-neighbor index (HNSW, in pgvector or a vector database).

The model is **all-MiniLM-L6-v2**: small (about 90 MB), widely used, and it runs locally on the CPU through ONNX Runtime. No embedding API, no cost per query, and SBG's content never leaves the server. The trade-off is lower quality than large hosted embedding models, which the hybrid search partly makes up for.

2.6 BM25: keyword search, done properly

Embeddings are fuzzy on exact tokens: "844-284-2725", "UCC", "7(a)", "Mission Capital LLC". **BM25** is the classic search-engine formula (it powers Elasticsearch and Lucene). For each query term it multiplies two factors:

- **IDF (inverse document frequency):** how rare the term is across all chunks. "UCC" appears in two chunks and matters a lot; "financing" appears everywhere and matters little.
- **Saturating term frequency:** more occurrences help, but with diminishing returns (parameter `k1 = 1.2`), discounted for long chunks (parameter `b = 0.75`) so they don't win just by being long.

```
score(D, Q) = Σ over terms t in Q:  IDF(t) × tf × (k1 + 1) / (tf + k1 × (1 − b + b × |D| / avgdl))
```

Before scoring, text is **tokenized** (`tokenize()` in `lib/rag/bm25.ts`): lowercase, "7(a)" becomes "7a", stopwords like "the" and "how" are dropped, and plurals are lightly stemmed so "invoices" matches "invoice".

2.7 Hybrid search and Reciprocal Rank Fusion

You have two ranked lists and their scores aren't comparable: cosine is 0–1, BM25 is 0 to about 15. **Reciprocal Rank Fusion** ignores scores and uses ranks:

```
rrf(d) = w_dense / (60 + dense_rank(d))  +  w_bm25 / (60 + bm25_rank(d))
```

`60` is the standard constant from the original paper; it flattens the advantage of being ranked first versus fifth. Sterling uses weights `dense = 1`, `bm25 = 0.3`, chosen by a sweep on the eval set (`npm run eval -- --sweep`). BM25 is a tiebreaker and a safety net, not an equal partner.

Worked example:

Chunk	Dense rank	BM25 rank	RRF
A	1	5	1/61 + 0.3/65 = 0.02101
B	3	1	1/63 + 0.3/61 = 0.02079
C	2	not in top 20	1/62 = 0.01613

A chunk both rankers like (A, B) beats one only dense search likes (C).

Confidence gate. The best cosine score decides confidence: 0.5 and above is high, 0.33 and above is medium, below that is low. Low means Sterling says it couldn't find the answer and offers a specialist. The thresholds were set by looking at the eval set: answerable questions almost always score above 0.45, and off-topic ones rarely pass 0.30.

2.8 Measuring retrieval: hit@k and MRR

The eval set (`evals/retrieval.json`) has 46 questions, each labeled with the chunk or chunks that answer it.

- **hit@k:** the share of questions where a correct chunk appears in the top k. hit@3 = 93.5% means 43 of 46 questions had the answer in the top three. hit@5 is what Claude sees, since each search returns five passages. hit@1 matters for retrieval-only mode, which quotes the top passage.
- **MRR (mean reciprocal rank):** for each question, 1 divided by the rank of the first correct chunk (rank 1 → 1, rank 2 → 0.5, rank 4 → 0.25, not found → 0), averaged over all questions. It rewards ranking the answer *first*.

Results:

	hit@1	hit@3	hit@5	MRR
Dense	73.9%	89.1%	93.5%	0.819
BM25	52.2%	71.7%	84.8%	0.637
Hybrid	73.9%	93.5%	95.7%	0.839

Be ready to discuss the misses, for example "Do you lend in Texas?": the knowledge base says "all 50 states" but never mentions Texas, so nothing matches well. Fixes: add a states passage, or rewrite queries before retrieval ("which states does SBG serve?").

2.9 Guardrails

Guardrails are checks around the model, not instructions inside it. Code guardrails can't be argued with.

On the way in (`checkInput` in `lib/engine/guardrails.ts`):

- **PII redaction.** SSNs (formatted, or 9 digits right after SSN wording), card numbers (13–19 digits that pass the **Luhn checksum**, the check digit every real card number satisfies, which cuts false alarms) and account numbers next to account wording. They're replaced before the text reaches the model or storage.
- **Routing.** Regex rules catch requests for a person, complaints or legal threats, payment hardship and account servicing. These skip the model entirely and go straight to the right path. Curly apostrophes (typed by phones) are normalized first.
- **Prompt-injection flag.** Phrases like "ignore previous instructions" are flagged in the trace. The real defense is structural, though: the model can't approve anything or change numbers because those

come from code, and the system prompt says retrieved text and user text are content, not commands.

On the way out (`checkOutput`): if an answer says "you're approved" or "guaranteed", a correction is appended. Text that has already streamed can't be unsent, so it's corrected visibly instead.

2.10 Human handoff

Escalation happens two ways:

- **By rule** (guardrails), for clear signals: asked for a person, complaint, hardship.
- **By the model** (`escalate_to_human`), for judgment calls: it can't answer, or a complex deal.

Both create one ticket per conversation (`lib/handoff/escalate.ts`). The ticket has a reason, a priority (complaints and hardship are high), the profile, whether the person meets the minimums, best-fit products (for sales leads only, never for hardship), the redacted transcript and a summary.

The summary is written in two stages. A template version is written instantly, so the ticket is useful even if the model is down. A model-written version replaces it *after* the response has been sent (Next.js `after()`), so the customer never waits for it.

The customer gets a reference code (`SBG-7KQ2MX`) from an alphabet without look-alike characters (no 0/O, no 1/I), so it can be read over the phone. Business hours are checked in New York time. Advisors see tickets at `/specialist` .

2.11 Graceful degradation

What fails	What happens
No API key	Retrieval-only mode: the same flow and engine; regex extraction instead of the model; answers quote the best passage
API error	Two automatic retries, then retrieval-only mode for that turn
Safety refusal	Server-side fallback to another model; if still refused, a polite redirect to the phone line
A bad tool input	Validation error goes back to the model to self-correct

2.12 Evals for routing, and their limits

`evals/routing.json` has 35 messages with the expected route, and all pass. Know the caveat: the set was written alongside the rules, so it shows the rules do what was intended, not how they'd do on real traffic. Six cases were added after a real bug: "I can't make this week's payment on my current advance" was routed to account servicing instead of hardship. Two causes: only one word was allowed before "payment", and curly apostrophes didn't match. Finding a failure, adding it to the eval set, then fixing it is exactly the loop you want.

3. Trace two requests through the code

Request A: clicking an answer card

1. `components/blocks/QuestionCard.tsx`: clicking "Payroll" calls `onAnswer("purpose", "payroll", "Payroll")`.
2. `components/chat/useChat.ts`, `send()`: adds the user bubble, POSTs `{state, input: {type: "answer", ...}}` to `/api/chat`, then reads the NDJSON stream line by line.
3. `app/api/chat/route.ts`: creates a `ReadableStream` and calls `handleTurn(request, emit, {defer: after})`.
4. `lib/chat/orchestrator.ts`, `handleTurn()`: `sanitizeState()` revalidates everything the client sent, then calls `handleAnswer()`.
5. `handleAnswer()`: `answerToUpdate()` converts the value (credit bands become a typical score, "yes" becomes `true`), `mergeProfile()` applies it, and a `profile` event goes out.
6. `advance()`: `nextQuestion(profile, intent)` in `lib/engine/flow.ts` returns the next unanswered field. Out goes a `question` UI block. When nothing is left: `evaluate()` produces the results and estimate cards.
7. `state` and `done` events close the turn. In the browser, `useChat` stores the new state (and in `sessionStorage`, so a refresh keeps the conversation).

No model call at all. Notice how much of the product is just good software.

Request B: typing a question (with a key set)

1. `handleText()` runs `checkInput()`: redaction, then routing. No rule fires.
2. `runAgent()` in `lib/agent/agent.ts` builds the messages: text history plus a final user message containing `<app_state>` (from `buildAppState()`) and the user's text.
3. `client.beta.messages.stream({...})`: the model, `max_tokens`, `thinking: {type: "adaptive"}`, `output_config: {effort: "low"}`, the cached system prompt, `tools: TOOLS`, `fallbacks: "default"`.
4. The stream loop: on `content_block_start` for a tool call, emit a status ("Searching SBG's knowledge base…"); on text deltas, emit `text`.
5. `finalMessage()`, then trace the model, stop reason, tokens and cache reads.
6. Tool calls go to `runTool()` in `lib/agent/tools.ts`, which zod-validates and dispatches:
   - `search_knowledge_base` → `search()` → numbered passages. Each passage is registered in `ctx.sources` so citations can be resolved later.
   - `update_business_profile` → `mergeProfile()` → a `profile` event with `via: "ai"`. That's what triggers the neon highlight in the funding file.
7. The results go back, the loop continues, and Claude writes the final answer with [n] citations.
8. `checkOutput()` (approval promises), then the orchestrator emits `sources` for the cited passages only, then `advance()` shows the next guided question.

Open "Under the hood" after sending a message and match each trace line to these steps.

---

4. The lending domain in one page

SBG Funding. New York, founded 2017, more than $3B funded for more than 28,000 businesses, $5K to $10M, all 50 states. It's a **direct lender** for revenue-based financing (through its balance-sheet arm, Net Funding) and a **marketplace** for everything else: it places term loans, lines of credit, SBA, equipment, invoice and bridge financing with partner lenders. One application, one dedicated advisor.

Published minimums. A 500+ personal FICO score, 6+ months in business, $150K+ annual revenue, and 4 recent business bank statements to apply. The business HELOC needs a 600+ score and has no revenue minimum. Applying uses a **soft credit pull**, which doesn't affect the score; a hard pull does.

The nine products, in one line each:

- **Revenue-based financing**: an advance repaid as a share of sales; payments flex with revenue. SBG funds it directly.
- **Term loan**: a lump sum repaid on a fixed schedule (6–48 months).
- **Line of credit**: revolving; draw, repay, draw again; interest only on what's drawn.
- **Working capital**: a lump sum for a defined operating gap, short term.
- **SBA 7(a)**: government-guaranteed, long terms (10–25 years), low rates, slow (30–90 days), more paperwork.
- **Equipment financing**: up to 90% of the equipment's value; the equipment is the collateral.
- **Invoice financing**: an advance of up to 90% of unpaid B2B invoices.
- **Bridge capital**: short-term money for a gap that ends with a known event, like a closing.
- **Business HELOC**: a credit line secured by home equity.

Vocabulary you'll hear:

- **Factor rate**: the total cost as a multiple. A 1.26 factor on $50,000 means repaying $63,000. Unlike APR it ignores the term, so the same factor over a shorter term is much more expensive per year.
- **APR**: annualized cost including fees. Lenders that quote factor rates make comparison harder; reviewers point this out about SBG.
- **UCC filing**: a public lien notice securing the lender's interest in business assets.
- **Personal guarantee**: the owner is personally liable if the business doesn't pay.
- **LTV (loan-to-value)**: the loan as a share of the asset's value (90% for equipment).
- **Stacking**: taking several advances at once. Lenders watch for it because it strains cash flow.

SBG's calculator model (ported in `lib/engine/estimator.ts`). Points for time in business (0–11), credit (0–11) and monthly revenue (1–14) add up to a score from 1 to 36. The score picks one of four tiers. Each tier has a revenue multiplier (0.75× to 2.5×), a factor rate (1.22–1.28), a term (12–36 months) and a payment frequency (weekly, twice-monthly or monthly). Below a score of 10 there's no automatic estimate, and the calculator points to a HELOC or an advisor.

Example: 2+ years (11 points) + 640 credit (3) + $50K/month (14) = 28 → tier 3. The maximum is 2.25 × $50K = $112,500, at a 1.26 factor over 24 months with twice-monthly payments. Pricing $50,000 instead: $63,000 to repay, in 48 payments of $1,310.

---

5. Study plan

About 3½ hours. Do the self-check at the end of each block out loud; if you can't answer one, reread that section.

Time	Do this	Self-check
0:00–0:20	Run the app. Click "Find the right financing" through to results. Type "I run a restaurant, LLC, 2 years, $60K a month". Ask for a person. Open `/specialist`. Open "Under the hood" after each step.	What happens on a button press vs. a typed message?
0:20–0:35	Read part 1 of this guide and `README.md`.	Explain "code decides, the model explains" with two examples.
0:35–1:15	Concepts 2.1–2.3, then read `lib/agent/agent.ts`, `lib/agent/tools.ts`, `lib/agent/prompt.ts`.	Draw the agent loop. Why validate tool inputs? Why is `<app_state>` not in the system prompt?
1:15–2:05	Concepts 2.4–2.8, then read `lib/rag/chunker.ts`, `lib/rag/bm25.ts`, `lib/rag/retriever.ts`. Run `npm run eval`.	Explain embeddings, cosine, BM25 and RRF without notes. Why hybrid? What do hit@3 and MRR measure?
2:05–2:35	Section 4, then `lib/engine/estimator.ts`, `lib/engine/eligibility.ts`, `lib/engine/flow.ts`. Skim `tests/estimator.test.ts`.	Work out the $50K example by hand. Why isn't eligibility decided by the model?
2:35–2:55	Concepts 2.9–2.12, then `lib/engine/guardrails.ts` and `lib/handoff/escalate.ts`.	Which messages never reach the model, and why? What's in a ticket?
2:55–3:20	`docs/ARCHITECTURE.md`, especially section 7. Draw the architecture from memory.	How would you take this to production? What breaks first at scale?
3:20–3:40	`docs/INTERVIEW_PREP.md`: rehearse the demo twice, out loud, with a timer.	Can you demo it in under three minutes?

Self-check answers

Why is `<app_state>` on the user message and not in the system prompt?

The system prompt is cached. Anything that changes per turn would invalidate the cache on every request. Static instructions go in the system prompt; dynamic context goes with the newest message.

Why not let the model decide eligibility?

Correctness (models slip on thresholds and arithmetic), consistency (same inputs, same answer, which matters for fair lending), testability (unit tests against known values) and change control (a minimum changes in one line of code, not in a prompt). The model explains the result; it doesn't produce it.

Why hybrid search instead of embeddings alone?

Embeddings capture meaning but blur exact tokens such as phone numbers, "7(a)", "UCC" and legal names. BM25 nails exact tokens but misses paraphrase. RRF combines their rankings without needing comparable scores. On the eval set hybrid lifts hit@3 from 89.1% to 93.5%.

What happens when retrieval finds nothing good?

If the best cosine is below 0.33, confidence is "low": Sterling says it couldn't find the answer in SBG's materials and offers a specialist instead of guessing. With the model running, the tool result tells it the confidence and the prompt says not to guess.

How does a phone number in a question get found, if embeddings are fuzzy on numbers?

BM25 keeps digits as tokens ("844", "284", "2725") and they're rare across chunks, so their IDF is high and the contact passage ranks first on the keyword side. RRF then lifts it.

Why is the server stateless, and what's the risk?

Any server instance can handle any turn, with no session store and no sticky sessions, so it scales horizontally and survives deploys. The risk is a tampered client state, so the server revalidates the whole state every turn and recomputes eligibility itself rather than trusting the client.

What's the difference between a soft and a hard credit pull?

A soft pull lets a lender see a credit report without affecting the score; SBG uses one to review applications. A hard pull is part of a formal credit application and can lower the score slightly. Some reviewers report a hard pull before funding; SBG's own site says soft pull only, so Sterling follows SBG and suggests confirming with an advisor.